

Windows绘图

党宇

dangyu@nankai.edu.cn



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南開大學

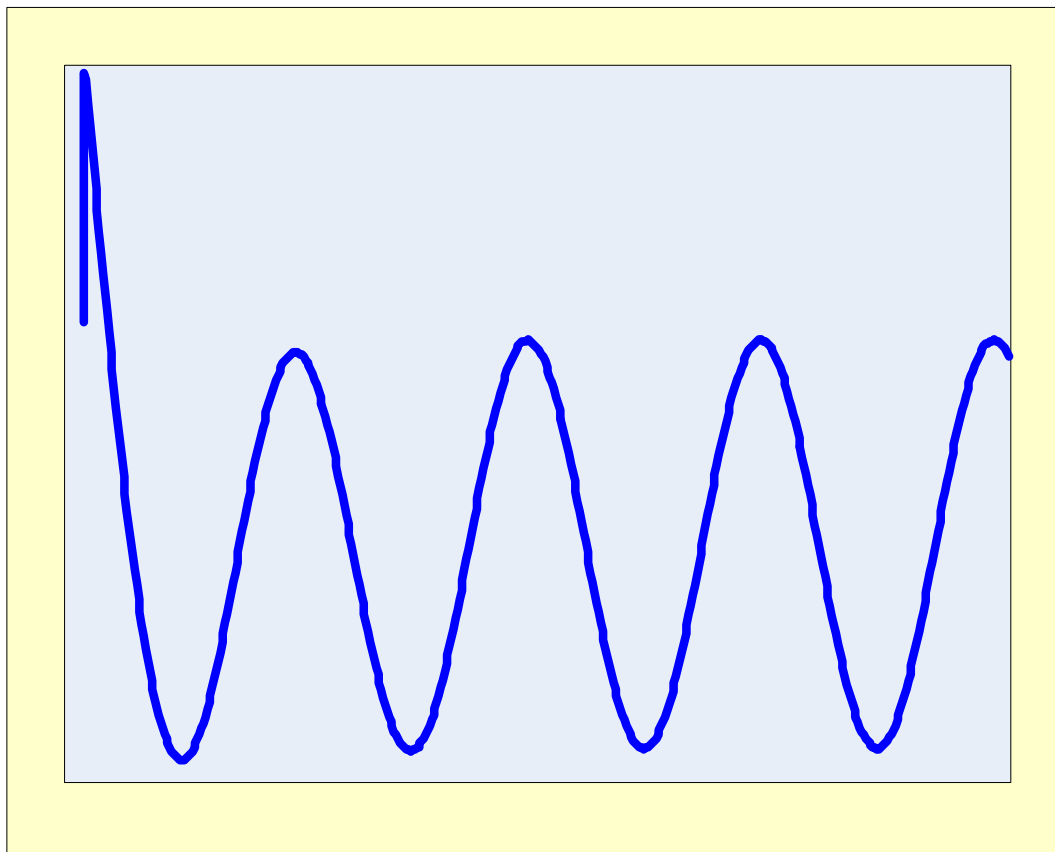
Nankai University

主要内容

- 主框架窗口与视图窗口
- GDI与设备环境
- 常用绘图函数
- 绘图工具
- 文本处理
- 位图操作
- 图标与光标
- 综合实例

综合实例1：根据数据绘制曲线

➤ 根据数据在客户区绘制曲线



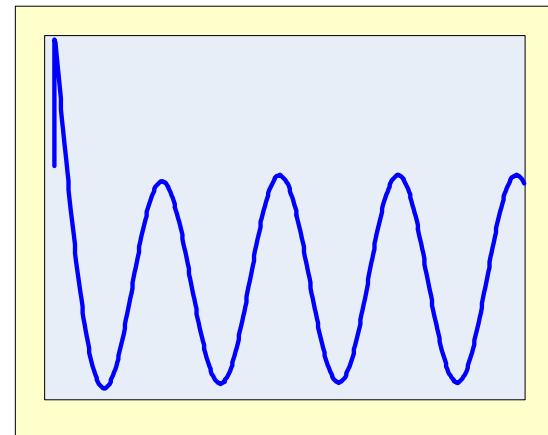
综合实例1：根据数据绘制曲线

➤ 分析

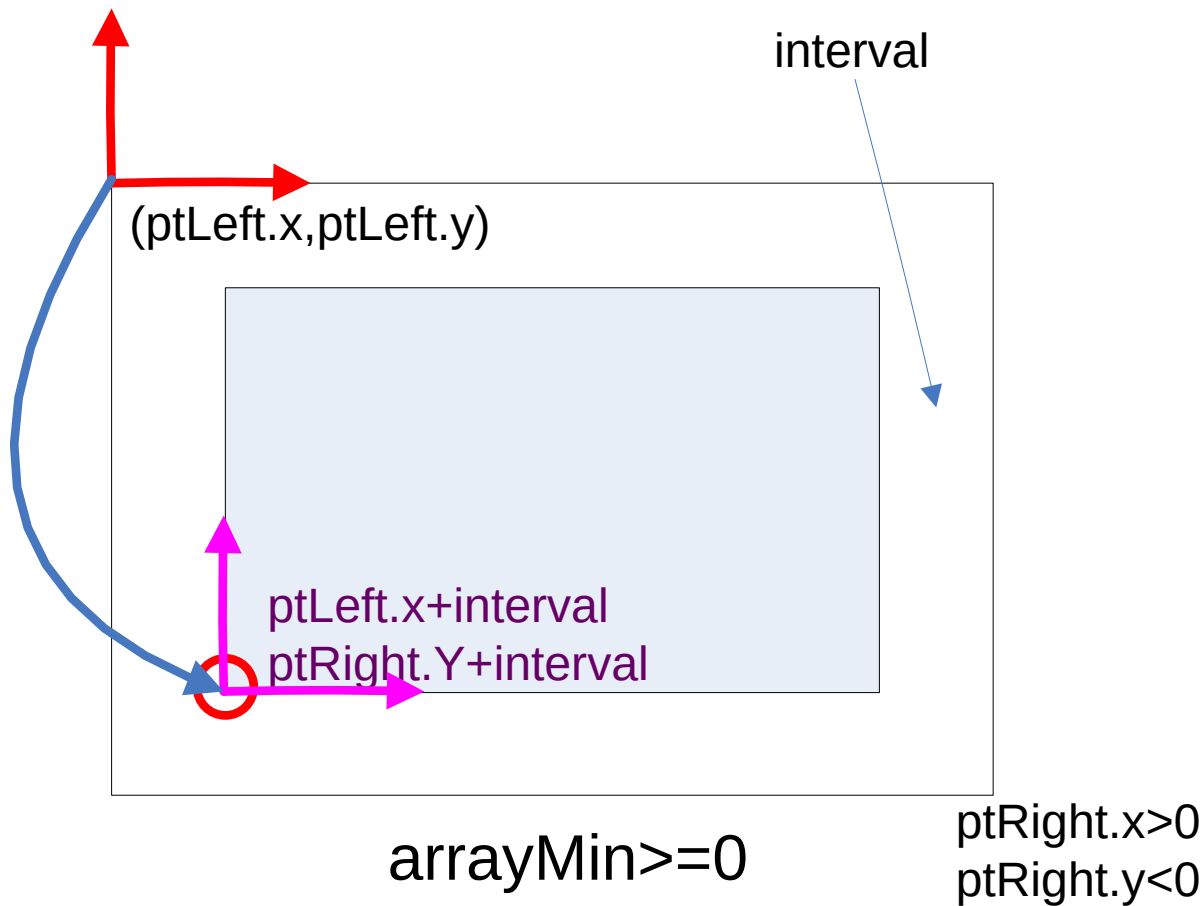
- 数据的获取：文件或程序
- 绘图函数：画线
- 绘图范围的获取：客户区
- 坐标系的设定：逻辑坐标系
- 如何使绘图结果根据视图窗口变化？

分别在x方向和y方向设定绘图比例，该比例可随视图窗口大小变化

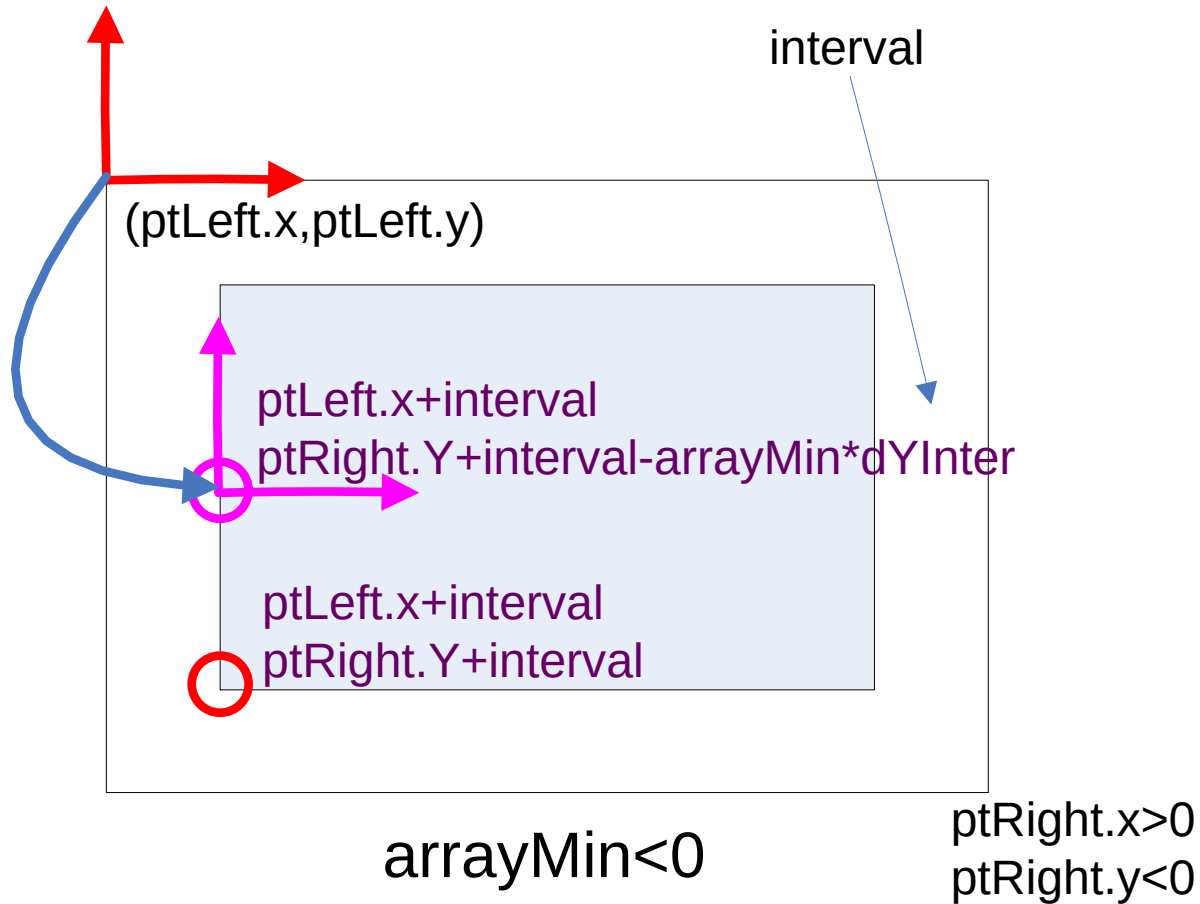
- 如何设置坐标系原点？



综合实例1：根据数据绘制曲线



综合实例1：根据数据绘制曲线



综合实例1：根据数据绘制曲线

➤ 分析绘图工作

- 0) 在OnDraw()函数内进行绘制
- 1) 获取绘图范围，设定映射模式（坐标系转换）
- 2) 根据实际数据，计算数据和绘图位置的比例关系，并重新设定坐标轴
- 3) 设定画笔，使用画线函数，实现绘制

综合实例1：根据数据绘制曲线

➤ 在C***View类定义中

```
static const int DATA_NUM = 500;           // 数据数目
private:
    double m_array[DATA_NUM];               // 需要绘制的数据
    int m_interval;                         // 绘图周边空白
```

➤ 在C***View.cpp中

```
#include <cmath>
#define PI 3.1415926
```

➤ 在C***View类构造函数中

```
for(int i=0; i<DATA_NUM; i++)
{
    m_array[i]=10*sin(i*2*PI/DATA_NUM);
}
m_interval = 50;
```


综合实例1：根据数据绘制曲线

- 在C***View类OnDraw()函数中
- 1) 获取绘图范围，设定映射模式

```
CRect rect;  
GetClientRect(&rect);  
  
pDC->SetMapMode(MM_LOMETRIC);           // 设置映射模式  
CPoint ptLeft(rect.left,rect.top);  
CPoint ptRight(rect.right,rect.bottom);  
  
pDC->DPtoLP(&ptLeft);                     // 设备坐标转换为逻辑坐标  
pDC->DPtoLP(&ptRight);
```

综合实例1：根据数据绘制曲线

- 2) 根据实际数据计算数据和绘图位置的比例关系，并重新设定坐标轴

```
// x方向数据和绘图位置的比例关系，留出两侧空白
double dXInter = (ptRight.x-ptLeft.x-m_interval*2)/((double)DATA_NUM);

// 获取数据最大最小值
double arrayMax = -1, arrayMin = 1000;
for(int i=0; i<DATA_NUM; i++) {
    if( m_array[i] > arrayMax ) arrayMax = m_array[i];
    if( m_array[i] < arrayMin ) arrayMin = m_array[i]; }

// y方向数据和绘图位置的比例关系，留出上下空白
double dYInter = (ptLeft.y-ptRight.y-m_interval*2)/(arrayMax-arrayMin);
// 重新设定坐标原点
pDC->SetWindowOrg(-m_interval-ptLeft.x,
                  (int)(-ptRight.y+arrayMin*dYInter-m_interval));
```

OnDraw
接上页

综合实例1：根据数据绘制曲线

➤ 3) 设定画笔，使用画线函数，实现绘制

```
CPen m_redPen;  
CPen * m_pOldPen;  
m_redPen.CreatePen(PS_SOLID,8,RGB(255,0,0));  
m_pOldPen=pDC->SelectObject(&m_redPen);  
  
int x,y;  
for(int i=0; i<DATA_NUM-1; i++) {  
    x = (int)(i*dXInter); y = (int)(m_array[i]*dYInter);  
    pDC->MoveTo(x,y);  
    x = (int)((i+1)*dXInter); y = (int)(m_array[i+1]*dYInter);  
    pDC->LineTo(x,y);  
}  
pDC->SelectObject(m_pOldPen);
```

OnDraw
接上页

综合实例2：通过鼠标画矩形

➤ 要求

- 通过鼠标在客户区绘制矩形
- 鼠标按下时改变光标形状
- 限制光标移动范围为客户区
- 绘制结束后，恢复原始光标形状

综合实例2：通过鼠标画矩形

➤ 在C***View类定义中

```
private:  
    bool m_isDraw;           // 是否绘制  
    HCURSOR m_Cursor;        // 改变的光标  
    CPoint m_ptStart;         // 矩形起点  
    CPoint m_ptNow;           // 矩形当前终点
```

➤ 在C***View构造函数中

```
m_isDraw=false;  
m_Cursor=AfxGetApp()->LoadStandardCursor(IDC_CROSS);
```

综合实例2：通过鼠标画矩形

➤ 在C**View类响应鼠标左键按下消息

```
m_ptStart = point;           // 保存当前鼠标位置为矩形起点
m_ptNow = point;             // 保存当前鼠标位置为矩形终点
m_isDraw=true;              // 开始绘制矩形

// 捕获鼠标，修改光标，限定鼠标移动范围
SetCapture();
SetCursor(m_Cursor);
CRect rect; GetClientRect(&rect);
ClientToScreen(&rect);
ClipCursor(&rect);
```

综合实例2：通过鼠标画矩形

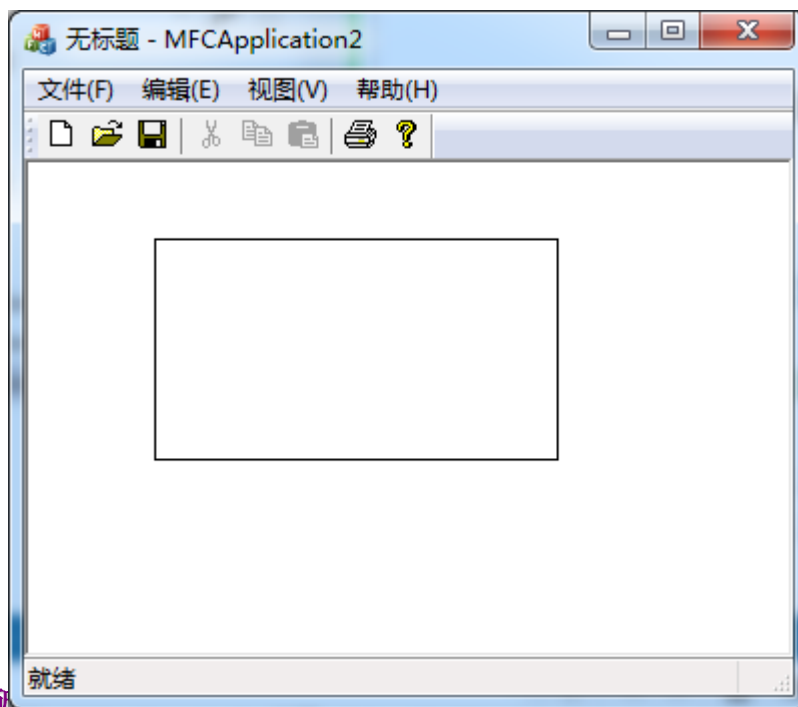
- 在C***View类响应鼠标左键移动消息(WM_MOUSE_MOVE)

```
CClientDC dc(this);
int nModeOld;
if(m_isDraw)
{
    dc.SelectStockObject(NULL_BRUSH);           // 设置画刷
    // 消去上次绘制矩形
    nModeOld = dc.SetROP2(R2_NOT);
    dc.Rectangle(m_ptStart.x,m_ptStart.y,m_ptNow.x,m_ptNow.y);
    // 绘制新的矩形
    dc.SetROP2(nModeOld);
    dc.Rectangle(m_ptStart.x,m_ptStart.y,point.x,point.y);
    m_ptNow=point;           // 保留当前鼠标位置为矩形新的终点
}
```

综合实例2：通过鼠标画矩形

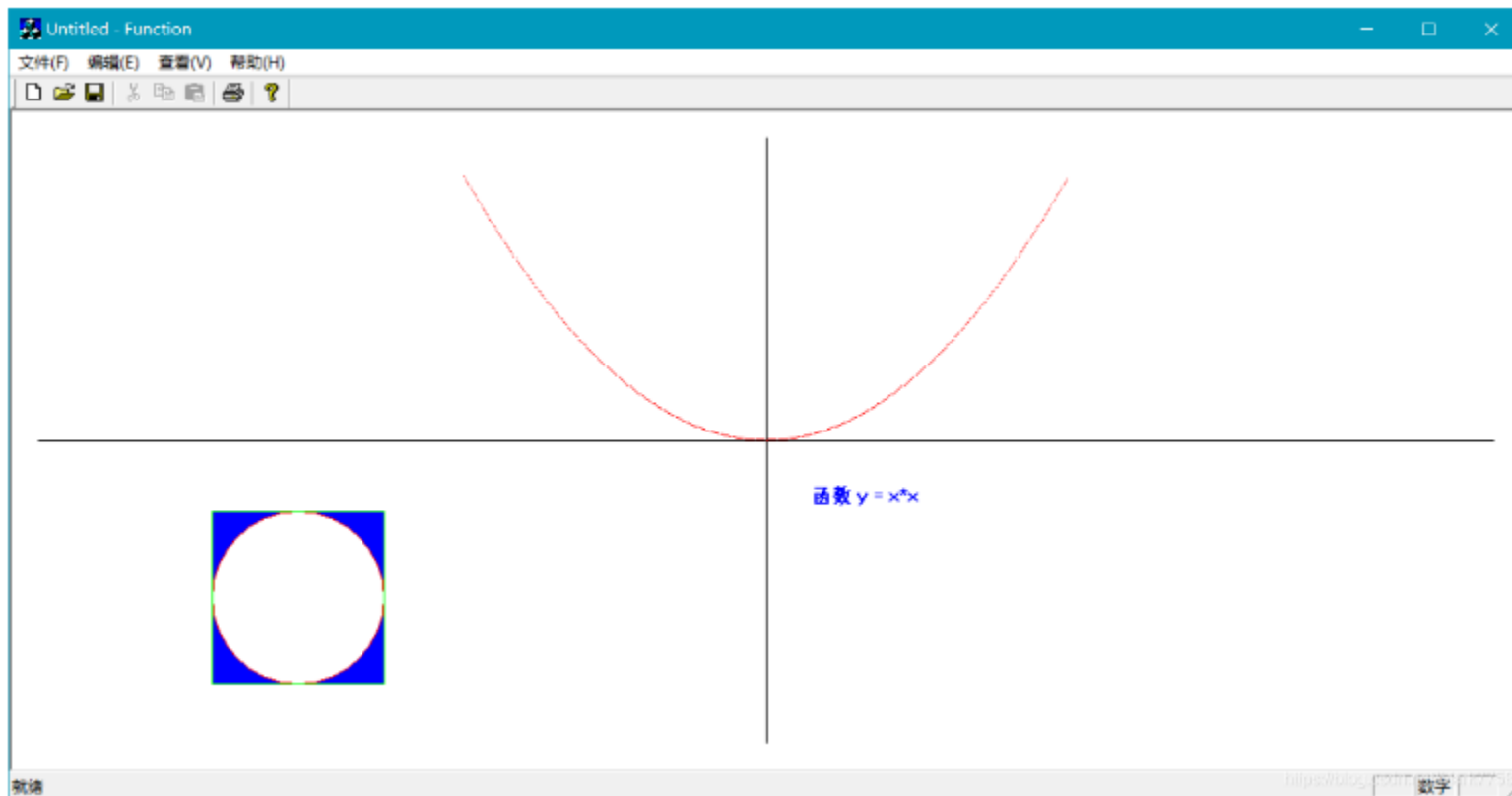
- 在C***View类响应鼠标左键抬起消息

```
m_isDraw=false;           // 停止绘制  
ReleaseCapture();          // 释放鼠标  
ClipCursor(NULL);
```



第五次课-课上练习

- 1. 以窗口中心点为原点，绘制 $y=x^2$ 的函数曲线，并在曲线旁边用文字描述，绘制刻度坐标。
- 2. 绘制一个红色的圆，并绘制该圆的外切正方形，该正方形填充颜色为蓝色，边线颜色为绿色。
- 3. 显示一幅位图-在菜单栏中找到插入，选择插入资源，类型为Bitmap。选择一张准备好的位图图片，选中插入。



总结

- 主框架窗口与视图窗口
- GDI与设备环境
- 常用绘图函数
- 绘图工具
- 文本处理
- 位图操作
- 图标与光标
- 综合实例